

Advanced Image Processing

Assignment 4 Report

Tejash More
SR: 27077

April 9, 2026

Question 1: Scalar Quantization Design

1.1 Introduction and Hyperparameters

In this part of the assignment, we designed and compared two types of quantizers: a Uniform Quantizer and a Lloyd-Max (non-uniform) Quantizer. Based on the provided code, the primary hyperparameter is the number of quantization levels, which is set to $N = 4$ (since there are 4 reconstruction points). The main objective is to minimize the distortion, measured as Mean Squared Error (MSE), between the original signal and the quantized signal.

1.2 Part 1: Uniform Quantization

For the uniform quantizer, the step size (Δ) is kept strictly constant between all adjacent decision boundaries. Running the optimization gave the following parameters:

- **Optimal Step Size (Δ):** 4.6134
- **Decision Boundaries (b):** $\{-\infty, -4.6134, 0.0, 4.6134, \infty\}$
- **Reconstruction Points (y):** $\{-6.9201, -2.3067, 2.3067, 6.9201\}$
- **Minimum Distortion (MSE):** 3.5334

1.3 Part 2: Lloyd-Max Quantization

Next, we used the Lloyd-Max algorithm. Unlike the uniform method, this algorithm iteratively updates the decision boundaries and reconstruction points to better fit the specific probability distribution of the input signal. The algorithm successfully converged in 11 iterations with the following results:

- **Decision Boundaries (b):** $\{-\infty, -4.7793, 0.0, 4.7793, \infty\}$

- **Reconstruction Points (y):** $\{-7.7793, -1.7805, 1.7805, 7.7793\}$
- **Minimum Distortion (MSE):** 3.1715

1.4 Analysis and Observations

Comparing the two methods, the Lloyd-Max quantizer achieved a noticeably lower MSE (3.1715) compared to the Uniform quantizer (3.5334). This results in an overall MSE reduction of 0.3619.

Looking at the result, we can see exactly why this improvement happens. The Uniform quantizer forces the step size to be equal everywhere ($\Delta = 4.6134$). However, the Lloyd-Max quantizer adapts to the shape of the data's distribution. It shifts the inner reconstruction points closer to zero (moving from ± 2.3067 to ± 1.7805) and pushes the outer points further away (moving from ± 6.9201 to ± 7.7793).

This behavior tells us that the input signal has a higher concentration of values near zero with longer outward tails. By making the inner intervals narrower, the Lloyd-Max algorithm reduces the error in the region where the data is most dense, heavily penalizing large errors in frequent values, which leads to the improved overall MSE.

Question 2: Comparison of Perceptual Quality Measures

In this problem, we evaluated how well different image quality metrics align with human perception. We compared Peak Signal-to-Noise Ratio (PSNR) against the Structural Similarity Index Measure (SSIM) and its individual components (Luminance, Contrast, and Structure). We used the provided dataset containing reference images and their Gaussian blurred versions, alongside the differential mean opinion scores (DMOS) which represent human judgments of quality degradation.

Methodology and Hyperparameters

The metrics were implemented from scratch in Python to ensure independent component evaluation. To compute the local statistics (mean, variance, and covariance) required for SSIM, I applied a Gaussian filter over the images. Based on standard implementations, the filter used a standard deviation of $\sigma = 1.5$ and was truncated at 3.5 standard deviations.

The constants used to stabilize the division in the SSIM formulas were derived using the standard hyperparameters $K_1 = 0.01$ and $K_2 = 0.03$. Since the images are standard 8-bit grayscale, the maximum pixel value used was $L = 255$.

As instructed, the original reference images were filtered out of the distorted set using the `blur_orgs` indicator, leaving a total of 145 distorted images. The metrics were computed for each image, and the Spearman Rank Order Correlation Coefficient (SROCC) was calculated against the DMOS scores. Because DMOS measures distortion (higher score

means worse quality) and our metrics measure quality (higher score means better quality), the raw correlation is negative. For clarity and easier comparison, the absolute values of the SROCC are reported below.

Results

The table below summarizes the absolute Spearman correlation values for each metric. A value closer to 1.0 indicates a stronger monotonic relationship with human perception (i.e., the metric ranks the image qualities similarly to how a human would).

Metric	Absolute SROCC
PSNR	0.7823
SSIM (Luminance)	0.9335
SSIM (Contrast)	0.9068
SSIM (Structure)	0.9055
Overall SSIM	0.9036

Table 1: Spearman Rank Order Correlation Coefficients

Observations

Looking at the extracted results and the bar plot, we can make a few clear observations:

- **PSNR performs the worst:** With an SROCC of 0.7823, PSNR is noticeably behind the others. This makes sense because PSNR only calculates the absolute pixel-wise errors (Mean Squared Error). It doesn't take into account how the human eye actually perceives structural shapes or local contrast.
- **SSIM is superior to PSNR:** All SSIM components, as well as the overall SSIM score, achieved an SROCC above 0.90. This confirms the theory that evaluating images based on luminance, contrast, and structure is much closer to human vision than pure pixel error.
- **Individual Component Performance:** Interestingly, for this specific blur dataset, the independent Luminance component had the highest correlation (0.9335) with the human scores, closely followed by Contrast and Structure.
- **Overall SSIM:** The overall SSIM (0.9036) performs very well, even if it is slightly lower than the isolated luminance term for this particular set of blurred images. Overall, the structural similarity approach proves to be a massive improvement over traditional PSNR for measuring perceptual quality.

1 Question 3: Object Detection and Real-World Robustness

1.1 (a) Model Training

For this task, I trained a YOLOv8 object detection model on the provided Vehicle Object Detection Dataset, which consists of 5 classes: bus, car, pickup, truck, and van.

To ensure fast and efficient training while managing memory, I utilized the YOLOv8 Nano architecture (`yolov8n.pt`). The model was trained using an NVIDIA A100-SXM4-80GB GPU. Given the large VRAM available, the AutoBatch feature was enabled, which automatically calculated an optimal batch size of 173.

The following key hyperparameters were used during the training phase:

- **Base Model:** YOLOv8 Nano (`yolov8n.pt`)
- **Epochs:** 15
- **Image Size (imgsz):** 640x640 pixels
- **Batch Size:** 173 (AutoBatch)
- **Learning Rate:** 0.001111
- **Momentum:** 0.9

The training completed successfully in approximately 0.38 hours, and the best weights were saved for the subsequent evaluation phases.

1.2 (b) Evaluation on Test Set

After training, the best model weights were evaluated on the held-out test set, which contained 3,716 images and 8,102 vehicle instances. The evaluation metrics used were Mean Average Precision at an Intersection over Union (IoU) threshold of 0.5 (**mAP@0.5IoU**), and the stricter average mAP across IoU thresholds from 0.5 to 0.95 (**mAP@[0.5:0.95]IoU**).

The overall performance on the test set was highly satisfactory, achieving an overall mAP@0.5 of **0.8920** and an overall mAP@[0.5:0.95] of **0.6993**. The class-wise breakdown is presented in Table 2.

Class	mAP@0.5IoU	mAP@[0.5:0.95]IoU
All (Overall)	0.892	0.699
Bus	0.880	0.631
Car	0.877	0.593
Pickup	0.919	0.717
Truck	0.911	0.757
Van	0.873	0.798

Table 2: Class-wise performance of the trained YOLOv8n model on the test set.

Observation: The model performs exceptionally well at finding the general location of the vehicles (as seen by the high mAP@0.5 scores across all categories). The ‘pickup’ and ‘truck’ classes had the highest detection accuracy. Interestingly, while the ‘van’ class had the lowest basic detection score (0.873), it had the highest strict score (0.798 for mAP@[0.5:0.95]), meaning that when the model does detect a van, the bounding box it draws is extremely tight and accurate.

Qualitative Results:

Figure 3 shows sample detections from the test set.



Figure 1: Sample bounding box predictions on the held-out test set.

1.3 (c) Domain Shift and Failure Analysis

To test the real-world robustness of the model, I captured several original photographs of vehicles around the university campus using my smartphone. These images were passed through the trained model to observe how it handles data that comes from outside its original training distribution.

Observations on Domain Shift:

1. **Lighting Conditions:** Harsh campus sunlight created deep shadows on the vehicles, confusing the feature extractor.
2. **Occlusion:** Vehicles parked on campus were often partially blocked by trees, bicycles, or pedestrians, which disrupted the physical shape the model was looking

for.

Figure 2 illustrates both a successful detection and a failure case from the campus environment.



(a) Successful detection despite complex background.



(b) Failure case due to severe occlusion/shadows.

Figure 2: Inference results on original smartphone photographs demonstrating domain shift.



Figure 3: Collection of images from campus